KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

INVENTORY PURCHASE ORDER MANAGEMENT SYSTEM

Submitted by

VARSHA.V

25BIT159

Under the Guidance of

MUGILAN.R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**INVENTORY PURCHASE ORDER MANAGEMENT SYSTEM"** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **MUGILAN.R**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

(VARSHA.V)

CERTIFICATE

This is to certify that the project entitled **"INVENTORY PURCHASE ORDER MANAGEMENT SYSTEM"** is a **bona fide work** carried out by **VARSHA.V B.SC.IT 'B'**, (2025-2026) [**INFORMATION TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Inventory Purchase Order System is a Java-based application designed to manage product inventory efficiently in small shops and businesses. Manual inventory management can lead to errors, stock mismanagement, and delays in processing orders. This system helps automate inventory operations by allowing users to add products, view available stock, and purchase items easily.

The system calculates the total cost of purchased products and updates the remaining stock automatically. It also identifies low stock items to help maintain proper inventory levels. The application is developed using Core Java concepts such as classes, objects, methods, loops, and conditional statements.

The main objective of this project is to improve inventory management, reduce human errors, and ensure accurate and efficient handling of purchase orders.

1. INTRODUCTION

In shops and small businesses, keeping track of products is very important for daily work. Many small stores still manage their stock manually using notebooks or simple records. This manual method takes more time and can cause mistakes, such as wrong calculations, missing records, or running out of stock without knowing.

With the help of computers, inventory management can be done faster and more accurately. A computerized inventory system helps store owners and workers easily add products, check available stock, and manage purchases. It also helps in keeping records in an organized way.

The Inventory Purchase Order System is a Java-based application developed to manage product inventory in a simple and efficient manner. The system allows the user to add product details such as product ID, product name, quantity, and price. Users can view the list of products, purchase items, and the system will automatically calculate the total cost and update the remaining stock. It also shows low stock products when the quantity becomes less than a certain limit.

This project is created using basic Core Java concepts like classes, objects, methods, loops, and conditions. The system runs as a console-based program and helps reduce manual work, avoid errors, and manage inventory easily.

1.1 Objective

The main objective of the Inventory Purchase Order System is to create a simple Java program that helps manage products and stock easily in shops or businesses.

The specific objectives of this project are:

- To create a system to add product details like ID, name, quantity, and price.
- To help users view the list of available products.
- To calculate the total cost automatically when a product is purchased.
- To reduce mistakes in managing stock manually.
- To check and show low stock products.
- To use basic Core Java concepts like classes, objects, methods, loops, and conditions.

1.2 Overview

The Inventory Purchase Order System is a menu-driven application developed using the Java programming language. The system provides a simple way to manage product details and stock in shops or small businesses. It helps users store product information and perform purchase operations easily using a computer.

The system allows the user to add products with details such as product ID, product name, quantity, and price. It displays the list of available products in the inventory. When a user purchases a product, the system asks for the quantity and automatically calculates the total cost and GST. After the purchase, the system updates the remaining stock quantity.

The system also checks for low stock items and displays products whose quantity is less than a certain limit. This helps the user know when to restock products. The application is developed as a console-based program using basic Core Java concepts.

This project shows how Java programming can be used to create simple applications for managing inventory in real-life business situations.

1.3 Features

Product Addition – Allows the user to add new products with ID, name, quantity, and price.

Product Display – Shows the list of all available products in the inventory.

Purchase Product – Enables the user to purchase a product by entering the required quantity.

Automatic Bill Calculation – Automatically calculates the total amount and GST during purchase.

Stock Update – Updates the product quantity after each purchase.

Low Stock Check – Displays products when the quantity is less than 5

User-Friendly Menu – Provides a simple menu-driven interface that is easy to use.

1.PROGRAM ANALYSIS

Program analysis is an important step in software development where the problem is studied carefully and the system requirements are identified. This stage helps to understand how the system should work and what features are needed to solve the problem effectively.

The analysis phase mainly focuses on identifying the problems in the existing manual system and providing a computerized solution that improves speed, accuracy, and efficiency.

2.1 Problem Definition

In many small shops and businesses, inventory management is done manually using notebooks or paper records. This traditional method creates several difficulties in managing products and stock.

Some common problems include:

•Errors in calculating product cost

•Difficulty in tracking available stock

•Time-consuming manual record keeping

•Risk of running out of stock without notice

•Poor management of purchase details

Manual inventory systems increase workload and may lead to mistakes in stock records.

•Therefore, there is a need for an automated inventory system that can manage products, calculate costs, and update stock quickly and accurately.

•The Inventory Purchase Order System is designed to solve these problems by providing a simple computerized solution for managing product inventory

2.2 Project Overview

The Inventory Purchase Order System is a Java-based application developed to make inventory management simple and efficient in shops and small businesses.

The system provides a menu-driven interface where users can add products, view available products, purchase items, and check low stock levels. When a product is purchased, the system automatically calculates the total cost and updates the remaining quantity.

The system includes the following functions:

- Adding new products with ID, name, quantity, and price
- Displaying the list of available products
- Purchasing products by entering quantity
- Calculating total cost and GST automatically
- Updating stock after purchase
- Checking and displaying low stock products
- This project is implemented using basic Core Java programming concepts and runs as a console-based program.

2.3 Scopes and Limitations

Scope

The scope of the Inventory Purchase Order System includes the following:

- Managing product details and stock
- Processing product purchases
- Calculating total cost automatically
- Updating inventory records
- Checking low stock levels
- Demonstrating the use of Java programming concepts

The system can be used in:

- Small shops
- Retail stores
- Supermarkets
- Warehouses
- Small business organizations

Limitation

- The system stores data only during program execution (no database).
- It supports a limited number of products in memory.
- It is a console-based system without a graphical interface.
- It does not support multiple users at the same time**.**
-

SYSTEM DESIGN

System design is the step where we plan how the program will work before writing the code. In this stage, the system is divided into small parts called modules. Each module performs a specific task. Proper system design makes the program easy to understand and use.

The Inventory Purchase Order System is a menu-driven program. It allows the user to add products, view products, purchase items, check low stock, and exit the program. The system automatically calculates the total cost and updates the stock after purchase

3.1 Modules

1. Add Product Module

This module is used to add new products to the inventory.

User enters:

Product ID

Product Name

Quantity

Price

Purpose:

To store product details in the system.

2. View Products Module

This module shows all products available in the inventory.

Purpose:

To display product details like ID, name, quantity, and price.

3. Purchase Product Module

This module is used to purchase a product.

User enters:

Product ID

Quantity to purchase

System will:

Check stock availability

Calculate total cost

Add GST

Update remaining quantity

Simple Formula:

Total = Quantity × Price

GST = 5% of Total

Final Bill = Total + GST

4. Check Low Stock Module

This module shows products with low quantity.

Condition:

If quantity is less than 5, the product is shown as low stock.

Purpose:

To know when to restock products.

5. Exit Module

This module is used to stop the program safely.

Purpose:

To close the application.

1,METHODOLOGY

Methodology explains the steps followed to develop the project. It shows how the system was planned, designed, created, tested, and documented.

For the Inventory Purchase Order System, simple steps were followed to make sure the program works correctly and manages products easily.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied carefully. The needs of shops and businesses are identified.

The main problems in manual inventory systems are:

- Mistakes in stock calculation
- Difficulty in tracking product quantity
- Time-consuming manual records
- Risk of running out of stock
- Poor stock management
- Based on these problems, an automated system was developed to manage products and calculate purchase cost easily.

4.2. System Design

In this stage, the structure of the system is planned before coding.

The system is divided into simple modules such as:

- Add Product Module
- View Products Module
- Purchase Product Module
- Check Low Stock Module
- Exit Module
- The program is designed as a menu-driven system so users can easily select options

4.3. Program Development

After designing the system, the program is developed using the Java programming language.

The following steps were followed during development:

- Creating the Product class
- Writing methods to add, view, and purchase products
- Implementing menu-driven logic
- Taking user input using Scanner
- Calculating total cost and GST
- Updating product quantity
- Basic Java concepts used:
- Classes and Objects
- ArrayList
- Methods
- Loops
- Conditional statements

4.4. Testing and Validation

Testing is done to make sure the program works correctly.

Different test cases were used to check the system.

Example Test Cases:

- Test Case
- Expected Result
- Add product
- Product added successfully
- View products
- Product list displayed
- Purchase product
- Total cost calculated correctly
- Low stock check
- Low stock items displayed
- Invalid product ID
- Product not found message shown
- This step ensures the program runs without errors and gives correct results.

4.5. Result and Documentation

After testing, the results of the system are recorded.

The system successfully:

•Stores product details

•Displays product list

•Calculates total purchase cost

•Updates stock quantity

•Shows low stock products

4.6. Algorithm

- Algorithm for Inventory Purchase Order System:
- Start the program.

- Display the menu options.
- Ask the user to choose an option.
- If user selects Add Product, enter product details.
- If user selects View Products, display product list.
- If user selects Purchase Product, enter product ID and quantity.
- Calculate total cost and GST.
- Update product quantity.
- If user selects Check Low Stock, display low stock products.
- Repeat steps until user selects Exit.
- End the program.

4.7. Flowchart

Start

↓

Display Menu

↓

User Choice

↓

Add / View / Purchase / Check Stock

↓

Display Result

↓

Exit

↓

End

1. SOURCE CODE

```
import java.util.ArrayList;
```

```
import java.util.Scanner;

class Product {
    int id;
    String name;
    int quantity;
    double price;

    Product(int id, String name, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
        this.price = price;
    }

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Quantity: " + quantity);
        System.out.println("Price: " + price);
        System.out.println("----------------------");
    }
}

public class InventorySystem {

    static ArrayList<Product> inventory = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);
```

```
    public static void main(String[] args) {

        int choice;

        do {
            System.out.println("\n===== INVENTORY PURCHASE ORDER SYSTEM
=====");
            System.out.println("1. Add Product");
            System.out.println("2. View Products");
            System.out.println("3. Purchase Product");
            System.out.println("4. Check Low Stock");
            System.out.println("5. Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    addProduct();
                    break;

                case 2:
                    viewProducts();
                    break;

                case 3:
                    purchaseProduct();
                    break;
```

```
            case 4:
                checkLowStock();
                break;

            case 5:
                System.out.println("Exiting system...");
                break;

            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 5);
}

static void addProduct() {
    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    sc.nextLine();

    System.out.print("Enter Product Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Quantity: ");
    int quantity = sc.nextInt();

    System.out.print("Enter Price: ");
    double price = sc.nextDouble();
```

```
    inventory.add(new Product(id, name, quantity, price));

    System.out.println("Product added successfully!");
}

static void viewProducts() {

    if (inventory.isEmpty()) {
        System.out.println("No products available.");
        return;
    }

    System.out.println("\n--- Product List ---");

    for (Product p : inventory) {
        p.display();
    }
}

static void purchaseProduct() {

    System.out.print("Enter Product ID to purchase: ");
    int id = sc.nextInt();

    boolean found = false;

    for (Product p : inventory) {
```

```
    if (p.id == id) {

        System.out.print("Enter quantity to purchase: ");
        int qty = sc.nextInt();

        if (qty <= p.quantity) {

            double total = qty * p.price;
            double gst = total * 0.05;
            double finalBill = total + gst;

            p.quantity -= qty;

            System.out.println("Purchase successful!");
            System.out.println("Total: " + total);
            System.out.println("GST (5%): " + gst);
            System.out.println("Final Bill: " + finalBill);

        } else {
            System.out.println("Not enough stock!");
        }

        found = true;
    }
}

if (!found) {
    System.out.println("Product not found!");
}
```

```
    }

    static void checkLowStock() {

        System.out.println("\n--- Low Stock Products (Quantity < 5) ---");

        for (Product p : inventory) {

            if (p.quantity < 5) {
                p.display();
            }

        }
    }
}
```

INPUT

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\KGCAS\Desktop\java>javac InventorySystem.java

C:\Users\KGCAS\Desktop\java>java InventorySystem

===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
Enter choice: 1
Enter Product ID: 101
Enter Product Name: pen
Enter Quantity: 10
Enter Price: 5
Product added successfully!

===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
Enter choice: 2

--- Product List ---
ID: 101
Name: pen
Quantity: 10
Price: 5.0
---------------------

===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
```

OUTPUT

```
C:/Windows/System32/cmd.e

--- Product List ---
ID: 101
Name: pen
Quantity: 10
Price: 5.0
---------------------

===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
Enter choice: 3
Enter Product ID to purchase: 101
Enter quantity to purchase: 5
Purchase successful!
Total: 10.0
GST (2%): 0.2
Final Bill: 10.2
===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
Enter choice: 4

--- Low Stock Products (Quantity < 2) ---

===== INVENTORY PURCHASE ORDER SYSTEM =====
1. Add Product
2. View Products
3. Purchase Product
4. Check Low Stock
5. Exit
Enter choice: 5
Exiting system...

C:/Users/KGCAS/Desktop/java>
```

7.DISCUSSION & CONCLUSION

7.1 Discussion

The Inventory Purchase Order System is developed to help manage products and stock in shops. The system allows the user to add products, view products, purchase items, and check low stock.

In this project, basic Java concepts like classes, objects, loops, and conditions were used. The system can calculate the total cost when a product is purchased and update the remaining quantity automatically.

The program was tested with different inputs, and it worked correctly. It helps reduce manual work and makes inventory management easier and faster.

7.2 Conclusion

The Inventory Purchase Order System is a simple and useful application developed using Java.

The system helps to manage product details, calculate purchase cost, and maintain stock easily. It reduces mistakes and saves time compared to manual methods.

This project helped us understand how Java can be used to create real-life applications. In the future, the system can be improved by adding a database and a graphical user interface (GUI).

8. REFERENCES

The following resources were used during the development of the Inventory Purchase Order System project:

Books

* Herbert Schildt – Java: The Complete Reference

* E. Balagurusamy – Programming with Java

Websites

* https://www.oracle.com/java

* https://www.geeksforgeeks.org

* https://www.w3schools.com

* https://www.javatpoint.com